

Practical Week-8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
lenovo@Manya$ mkdir -p ~/DynamicMemory_Project
lenovo@Manya$ cd ~/DynamicMemory_Project
lenovo@Manya~/DynamicMemory_Project$ pwd
/home/akshitha_reddy/DynamicMemory_Project
lenovo@Manya~/DynamicMemory_Project$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

lenovo@Manya~/DynamicMemory_Project$ valgrind --version
valgrind-3.26.0
lenovo@Manya~/DynamicMemory_Project$ nano memory_alloc.c
lenovo@Manya~/DynamicMemory_Project$ gcc -Wall -Wextra memory_alloc.c -o memory_alloc
lenovo@Manya~/DynamicMemory_Project$ ls
memory_alloc  memory_alloc.c
lenovo@Manya~/DynamicMemory_Project$ ./memory_alloc
Dynamic Memory Allocation Demo
-----

malloc(): Allocated memory for 5 integers
malloc values: 10 20 30 40 50

calloc(): Allocated memory for 5 integers
calloc values: 0 0 0 0 0

realloc(): Increased arr1 from 5 to 10 integers
After realloc: 10 20 30 40 50 60 70 80 90 100

free(): arr1 memory released
free(): arr2 memory released

Intentional memory leak created.
Program finished.
lenovo@Manya~/DynamicMemory_Project$ |
```

```
lenovo@Manya × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("Dynamic Memory Allocation Demo\n");
    printf("-----\n\n");

    // 1. malloc()
    int *arr1 = (int *)malloc(5 * sizeof(int));

    if (arr1 == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    printf("malloc(): Allocated memory for 5 integers\n");

    for (int i = 0; i < 5; i++)
    {
        arr1[i] = (i + 1) * 10;
    }

    printf("malloc values: ");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr1[i]);
    }
    printf("\n\n");

    // 2. calloc()
    int *arr2 = (int *)calloc(5, sizeof(int));

    if (arr2 == NULL)
    {
```

GNU nano 8.7.1

```
if (arr2 == NULL)
{
    printf("calloc failed\n");
    free(arr1);
    return 1;
}

printf("calloc(): Allocated memory for 5 integers\n");
printf("calloc values: ");

for (int i = 0; i < 5; i++)
{
    printf("%d ", arr2[i]);
}
printf("\n\n");

// 3. realloc()
arr1 = (int *)realloc(arr1, 10 * sizeof(int));

if (arr1 == NULL)
{
    printf("realloc failed\n");
    free(arr2);
    return 1;
}

printf("realloc(): Increased arr1 from 5 to 10 integers\n");

for (int i = 5; i < 10; i++)
{
    arr1[i] = (i + 1) * 10;
}

printf("After realloc: ");

for (int i = 0; i < 10; i++)
```

```
for (int i = 0; i < 10; i++)
{
    printf("%d ", arr1[i]);
}

printf("\n\n");

// 4. free()
free(arr1);
printf("free(): arr1 memory released\n");

free(arr2);
printf("free(): arr2 memory released\n");

// Intentional memory leak
int *leak = (int *)malloc(100 * sizeof(int));

if (leak != NULL)
{
    printf("\nIntentional memory leak created.\n");
}

printf("Program finished.\n");

return 0;
}
```

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
lenovo@Manya$ mkdir -p ~/COW_Project
lenovo@Manya$ cd ~/COW_Project
lenovo@Manya~/COW_Project$ nano cow.c
lenovo@Manya~/COW_Project$ gcc -Wall -Wextra cow.c -o cow
lenovo@Manya~/COW_Project$ ls
cow  cow.c
lenovo@Manya~/COW_Project$ ./cow
Copy-on-Write Demonstration
-----

Parent PID: 1284

Parent memory BEFORE fork:
5d7ac03a9000-7ffffcb9bf000 ---p 00000000 00:00 0 [rollup]
Rss:          53008 kB
Pss:          51378 kB
Pss_Dirty:    51308 kB
Pss_Anon:     51296 kB
Pss_File:      82 kB
Pss_Shmem:     0 kB
Shared_Clean:  1700 kB
Shared_Dirty:  0 kB
Private_Clean: 0 kB
Private_Dirty: 51308 kB
Referenced:   53008 kB
Anonymous:    51296 kB
KSM:          0 kB
LazyFree:     0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap:         0 kB
SwapPss:      0 kB
Locked:       0 kB
```



lenovo@Manya



Child PID: 1285

Child memory immediately AFTER fork:

5d7ac03a9000-7ffffcb9bf000 ---p 00000000 00:00 0

[rollup]

Rss:	52200	kB
Pss:	25699	kB
Pss_Dirty:	25666	kB
Pss_Anon:	25662	kB
Pss_File:	37	kB
Pss_Shmem:	0	kB
Shared_Clean:	896	kB
Shared_Dirty:	51276	kB
Private_Clean:	0	kB
Private_Dirty:	28	kB
Referenced:	960	kB
Anonymous:	51296	kB
KSM:	0	kB
LazyFree:	0	kB
AnonHugePages:	0	kB
ShmemPmdMapped:	0	kB
FilePmdMapped:	0	kB
Shared_Hugetlb:	0	kB
Private_Hugetlb:	0	kB
Swap:	0	kB
SwapPss:	0	kB
Locked:	0	kB

Child is modifying the shared memory...

Child memory AFTER modification:

5d7ac03a9000-7ffffcb9bf000 ---p 00000000 00:00 0

[rollup]

Rss:	52264	kB
Pss:	51302	kB
Pss_Dirty:	51266	kB
Pss_Anon:	51262	kB
Pss_File:	40	kB
Pss_Shmem:	0	kB
Shared_Clean:	960	kB
Shared_Dirty:	76	kB
Private_Clean:	0	kB
Private_Dirty:	51228	kB
Referenced:	52228	kB
Anonymous:	51296	kB
KSM:	0	kB
LazyFree:	0	kB
AnonHugePages:	0	kB
ShmemPmdMapped:	0	kB
FilePmdMapped:	0	kB
Shared_Hugetlb:	0	kB
Private_Hugetlb:	0	kB
Swap:	0	kB
SwapPss:	0	kB
Locked:	0	kB

```
Child data[0] = B

Parent checking its data...
Parent data[0] = A

Parent memory AFTER child modification:
5d7ac03a9000-7ffffcb9bf000 ---p 00000000 00:00 0 [rollup]
Rss:                53072 kB
Pss:                51381 kB
Pss_Dirty:          51308 kB
Pss_Anon:           51296 kB
Pss_File:           85 kB
Pss_Shmem:           0 kB
Shared_Clean:        1764 kB
Shared_Dirty:         0 kB
Private_Clean:        0 kB
Private_Dirty:       51308 kB
Referenced:          53072 kB
Anonymous:           51296 kB
KSM:                 0 kB
LazyFree:            0 kB
AnonHugePages:        0 kB
ShmemPmdMapped:       0 kB
FilePmdMapped:        0 kB
Shared_Hugetlb:       0 kB
Private_Hugetlb:      0 kB
Swap:                0 kB
SwapPss:              0 kB
Locked:              0 kB
akshitha_reddy@Akshitha-Reddy:~/COW_Project$
```

```
lenovo@Manya × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (50 * 1024 * 1024)

void show_memory(const char *message)
{
    FILE *file;
    char line[256];

    printf("\n%s\n", message);

    file = fopen("/proc/self/smaps_rollup", "r");

    if (file == NULL)
    {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), file))
    {
        if (sscanf(line, "Rss:") == 0 ||
            sscanf(line, "Pss:") == 0)
        {
            printf("%s", line);
        }
    }

    fclose(file);
}

int main()
{
    setbuf(stdout, NULL);

    printf("Copy-on-Write Demonstration\n");
    printf("-----\n");

    // Allocate 50 MB
    char *data = malloc(SIZE);

    if (data == NULL)
    {
```



```

{
    perror("malloc");
    return 1;
}

// Touch every page so physical memory is actually allocated
for (size_t i = 0; i < SIZE; i += 4096)
{
    data[i] = 'A';
}

printf("\nParent PID: %d\n", getpid());

show_memory("Parent memory BEFORE fork:");

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    free(data);
    return 1;
}

if (pid == 0)
{
    // Child process
    printf("\nChild PID: %d\n", getpid());

    show_memory("Child memory immediately AFTER fork:");

    printf("\nChild is modifying the shared memory...\n");

    // Modify every page
    for (size_t i = 0; i < SIZE; i += 4096)
    {
        data[i] = 'B';
    }

    show_memory("Child memory AFTER modification:");

    printf("\nChild data[0] = %c\n", data[0]);

    free(data);

    exit(0);
}

```

```
        exit(0);
    }
    else
    {
        // Parent process
        wait(NULL);

        printf("\nParent checking its data...\n");

        printf("Parent data[0] = %c\n", data[0]);

        show_memory("Parent memory AFTER child modification:");

        free(data);
    }

    return 0;
}
```